

F0 — Validação de Execução do App Gerado (Quântica Comercial)

Data: 22/07/2026 · Sessão de código: `af4708e5` (55 arquivos) LLM: qwen2.5-coder-32b-instruct (LM Studio local, 192.168.1.115) · Banco do app: `quantica_ops`

Veredito

✅ O aplicativo gerado RODA de verdade, ponta a ponta. Os dois caminhos de execução funcionam e o frontend compila. A geração do LangNet produz um sistema **executável**, não apenas arquivos plausíveis.

Ambiente montado

Item	Ação
Extração	55 arquivos da sessão <code>af4708e5</code> gravados em <code>~/quantica-app-gerada/</code>
Banco do app	<code>quantica_ops</code> (banco separado do <code>langnet</code> — sem risco ao LangNet); 20 tabelas já existentes
<code>.env</code> do ws-server	<code>LLM_PROVIDER=lmstudio</code> , modelo <code>qwen2.5-coder-32b-instruct</code> , <code>DB_NAME=quantica_ops</code>
Dependências	<code>crewai</code> , <code>websockets</code> , <code>mysql-connector</code> , <code>litellm</code> , <code>pyyaml</code> , <code>dotenv</code> — todas presentes

Testes executados e resultados

#	Teste	Resultado	Evidência
T-F0.1	Subir <code>ws-server</code> (main.py) na porta 5002	✅ PASSOU	Porta 5002 ativa; handshake <code>connected</code> lista 18 tasks
T-F0.2	Task determinística <code>cadastrear_pessoa_alvo</code> → INSERT real	✅ PASSOU	<code>task_completed {status:sucesso, pessoa_id:UUID}</code> ; no banco: pessoa + 2 canais + 2 problemas + gatilhos/objeções/palavras corretos
T-F0.3	Task CrewAI <code>verificar_fatos_revisor</code> → <code>crew.kickoff()</code> no LLM local	✅ PASSOU	~30s; <code>task_completed</code> com <code>{"status":"falha"}</code> (o verificador julgou "300% em 2 dias" como não-comprovável — avaliação correta). Zero exceptions no log.
T-F0.4	<code>npm install</code> + <code>npm run build</code> do frontend	✅ PASSOU	1302 pacotes; "The build folder is ready to be deployed"; bundle 228 KB; só warnings <code>no-unused-vars</code>

Conclusão dos testes: a cadeia `tela` → `runTask` → `WebSocket :5002` → `ws-server` → (SQL determinístico OU CrewAI+LLM) → **resultado** está **funcional**. O banco recebe escritas reais; o LLM local executa tarefas cognitivas; o frontend compila para produção.

Achados / problemas (com severidade)

Sev.	Achado	Detalhe	Onde corrigir
Média	<code>.env.example</code> vaza a senha real do banco	Vem com <code>DB_PASSWORD=112358123</code> (produção) e default <code>LLM_PROVIDER=deepseek</code> (nuvem, que o usuário rejeita)	Gerador: usar placeholders (<code>DB_PASSWORD=</code>) e default local/seguro
Média	Nome do modelo LM Studio errado no template	<code>.env.example</code> traz <code>LMSTUDIO_MODEL_NAME=openai/deepseek-r1-distill-qwen-32b</code> , não o modelo carregado	Gerador: parametrizar pelo modelo real do projeto
Baixa	Adapter determinístico não trata input string	Se <code>canais</code> chega como <code>"a, b"</code> (string), o INSERT itera caractere a caractere . No fluxo real a tela usa <code>splitlist</code> (envia lista) e funciona; falta defesa	<code>ws-server/adapters.py</code> (gerador): normalizar string → lista no <code>_deterministic</code>
Baixa	Algumas telas não disparam task	Ex.: <code>GerenciarPermissoesUsuarios.jsx</code> importa <code>runTask</code> / <code>onPrimary</code> mas não os usa (form estático)	Gerador <code>_generate_business_screens</code> : garantir binding ação → task em todos os tipos
Baixa	Saída de task cognitiva é enxuta	<code>verificar_fatos_revisor</code> devolveu só <code>{"status":"falha"}</code> ; poderia trazer justificativa/campos	Prompt da task / <code>output_func</code> no gerador

Sev.	Achado	Detalhe	Onde corrigir
● Info	Schema real não está em <code>data_model_sessions.schema_sql</code>	A coluna trazia stub "teste"; as tabelas vêm de <code>entities_json</code> . O <code>quantica_ops</code> já tinha o schema completo de uma execução anterior	Verificar por que <code>schema_sql</code> ficou stub nessa sessão

Correções recomendadas no GERADOR (não no artefato)

1. **Segurança do `.env`** (●): emitir placeholders para segredos e default de LLM local; nunca embutir a senha real do banco. *Teste:* `grep` no `.env.example` gerado não encontra senha real.
2. **Parametrizar o modelo LM Studio** (●) pelo modelo configurado no projeto. *Teste:* `.env.example` traz o modelo correto.
3. **Robustez do adapter determinístico** (●): normalizar campos de lista (`splitList` no servidor). *Teste:* `cadastrar_persona_alvo` com `canais` string cria 1 canal "a, b" (ou 2), nunca N caracteres.
4. **Binding ação → task em todas as telas** (●). *Teste:* nenhuma tela gerada tem `runTask` importado e não usado.

Conclusão

O **F0 confirma a hipótese central da auditoria**: o pipeline do LangNet gera um app **que executa** — telas React reais, persistência SQL real e agentes CrewAI reais rodando no LLM local. Os problemas encontrados são **de polimento do gerador** (segurança do `.env`, defesa de input, binding de telas), não falhas estruturais. O caminho está aberto para as próximas fases (F1 — Configurações reais, etc.).

Como reproduzir: `cd ~/quantica-app-gerada/ws-server && python3 main.py` (porta 5002) e disparar `execute_task` via WebSocket; frontend em `~/quantica-app-gerada/frontend` com `npm start` (porta 3001).